

This process allows the cluster to somehow self-heal after an external event that caused its normal life cycle to abort. This makes it clear that the main aim of the WALs is not to avoid performance degradations but, rather, to ensure the cluster is able to recover after a crash. And in order to be able to do that, it must have data written permanently to the storage, but thanks to the sequential way in which WALs are written, data is made persistent with less I/O penalties.

Checkpoints

Sooner or later, the cluster must make every change that has already been written in WALs also available in the data files; that is, it has to write tuples in an I/O-scattered way. These writes happen at very specific times, named **checkpoints**. A checkpoint is a point in time at which a database makes an extra effort to ensure that everything already present in the WALs is also written in the correct position in the data storage.

The following diagram helps to understand what happens during a CHECKPOINT:

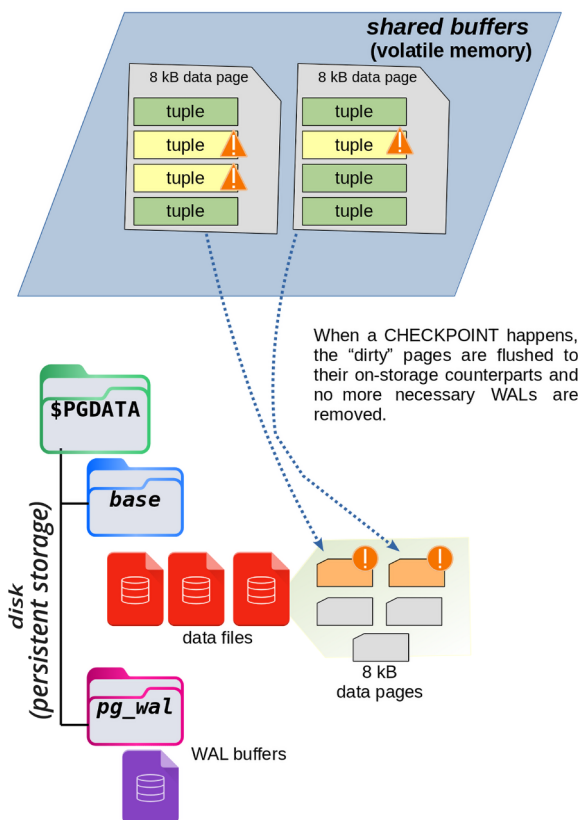


Figure 11.4: An example of a CHECKPOINT

But why should the database make this synchronization effort?

If the synchronization does not happen, the WALs will be the only source containing changes between the in-memory situation and the on-disk one, and therefore, they will keep growing and consuming storage space. Moreover, if the database crashes for any reason, the WAL replay must investigate a very long set of WALs.

Thanks to checkpoints, instead, the cluster knows that in the event of a crash, it has to synchronize data between the storage and the WALs only after the last checkpoint is successfully performed. In other words, both the storage space and time required to replay the WALs are reduced from the crash instant to the last checkpoint.

However, there is another advantage: since after a checkpoint PostgreSQL knows that the data in the WALs has been synchronized with the data in the storage, it can throw away already synchronized WALs. In fact, even in the event of a crash, PostgreSQL will not need any WAL part that precedes the last checkpoint at all. Therefore, PostgreSQL performs *WAL recycling*: after a checkpoint, a WAL segment is reused as an empty segment for the upcoming changes.

Thanks to this machinery, the space required to store WAL segments will pretty much remain the same during the cluster life cycle because, at every checkpoint, segments will be reused. Most notably, in the event of a crash, the number of WAL segments to replay will be the total number of those produced since the last checkpoint.



PostgreSQL is able to provide you with some information about how many WAL segments a specific query will consume, that is, how much data is inserted into the WALs due to the execution of a query. The special command `EXPLAIN` (detailed in *Chapter 13, Indexes and Performance Optimization*) can provide you with the WAL information. Moreover, it is possible to query special catalogs and even logs to get information about checkpoints and the quantity of generated and recycled WALs.

Checkpoint configuration parameters

The database administrator can fine-tune the checkpoints, meaning they can decide when and how often a checkpoint can happen. Since checkpoints are consolidating points, the more often they happen, the less time will be required to recover if there is a crash. On the other hand, executing checkpoints continuously will require I/O resources and could slow down your database system.

In fact, when a checkpoint is reached, the database must force every dirty buffer from memory to disk, and this usually means that an I/O spike is introduced; during such a spike, other concurrent database activities, such as getting new data from the storage, will be penalized because the I/O bandwidth is temporarily exhausted from the checkpoint activity.

For the preceding reasons, it is very important to carefully tune checkpoints, and in particular, their tuning must reflect the cluster workload.

Checkpoints can be tuned by means of three main configuration parameters that interact with each other, which are explained in the following subsections.

checkpoint_timeout and max_wal_size

Checkpoint frequency can be tuned by two orthogonal parameters: `max_wal_size` and `checkpoint_timeout`.

The `max_wal_size` parameter dictates how much space the `pg_wal` directory can occupy. Since at every checkpoint WAL segments are recycled, the `pg_wal` directory tends to occupy the very same size eventually. Tuning the `max_wal_size` parameter specifies after how many data changes the checkpoint must be completed, and therefore, this parameter is a *quantity* specification.

`checkpoint_timeout` expresses after how much time the checkpoint must be forced.

The two parameters are orthogonal, meaning that the first that happens triggers the checkpoint execution; your database produces data changes over the `max_wal_size` parameter or when the `checkpoint_timeout` time has elapsed.

As an example, let's take a system with the default settings:

```
forumdb=> SHOW checkpoint_timeout;
checkpoint_timeout
-----
5min
(1 row)

forumdb=> SHOW max_wal_size ;
max_wal_size
-----
1GB
(1 row)
```

```
-- or you can query the pg_settings
forumdb=> SELECT name, setting, unit
          FROM pg_settings
          WHERE name IN ( 'checkpoint_timeout', 'max_wal_size' );
name      | setting | unit
-----+-----+-----
checkpoint_timeout | 300     | s
max_wal_size      | 1024    | MB
(2 rows)
```

After 300 seconds (5 minutes) a checkpoint is triggered unless, in the meantime, 1,024 MB of WAL data has been produced. The aim of these two parameters is to guarantee that there will be neither too much time nor changed data between two consecutive checkpoints, thereby reducing the difference between the situation of data in memory from what is safely stored on the disk.

Since WAL segments are populated by writing transactions, the setting `max_wal_size` is proportional to the amount of user data changed into the database. The amount of data produced into the WALs is not exactly the amount of user data produced, since the WAL segments also store checksums and other information useful for the database in the case of a recovery or a replica, but you can approximate that the amount of WAL produced reflects the amount of database changed data.

Therefore, the meaning of `max_wal_size` is that once the database has produced such amounts of data changes, the checkpoint happens, and the in-memory situation is reflected and synced to disk. On the other hand, if your database is not busy and thus is not producing enough data to hit `max_wal_size`, the `checkpoint_timeout` is triggered, meaning that you are assured that even low loads will be synced after a specific amount of time.

In the default configuration, shown above, a checkpoint will be issued every 5 minutes or once 1 GB of new (WAL) data has been produced.

Please note that a checkpoint happens only if the situation has changed since the last checkpoint; in the rare case that a database has not produced any new data (i.e., it is not loaded at all), a checkpoint will not be triggered even if the `checkpoint_timeout` time has elapsed. In fact, if no data has been changed since the last checkpoint, there is nothing to synchronize and, therefore, no work at all to do by means of a checkpoint.

Checkpoint throttling

There is no urge to complete a checkpoint: if a system does not synchronize all the in-memory data on disk, and a failure occurs, the system will replay all the WAL segments to the previous completed checkpoint. On the basis of this consideration, PostgreSQL provides a configuration parameter that instructs the system on how fast a checkpoint must be completed.

In order to avoid an I/O spike at the execution of a checkpoint, PostgreSQL uses the `checkpoint_completion_target`, which can handle values between 0 and 1. This parameter indicates the amount of time the checkpoint can delay the writing of dirty buffers. In particular, the time provided to complete a checkpoint is computed as `checkpoint_timeout x checkpoint_completion_target`.

For example, if `checkpoint_completion_target` is set to 0.2 and `checkpoint_timeout` is 300 seconds, the system will have 60 seconds to write all the data. The system calibrates the required storage I/O bandwidth to fulfill the dirty buffers' writing.

The more you set `checkpoint_completion_target` close to 0, the more you will see I/O spikes at checkpoint execution times, with the consequence of high usage of I/O bandwidth; on the other hand, setting the parameter close to 1 will avoid I/O spikes producing, instead, a continuous I/O activity with low bandwidth consumption. The following picture illustrates the above concept:

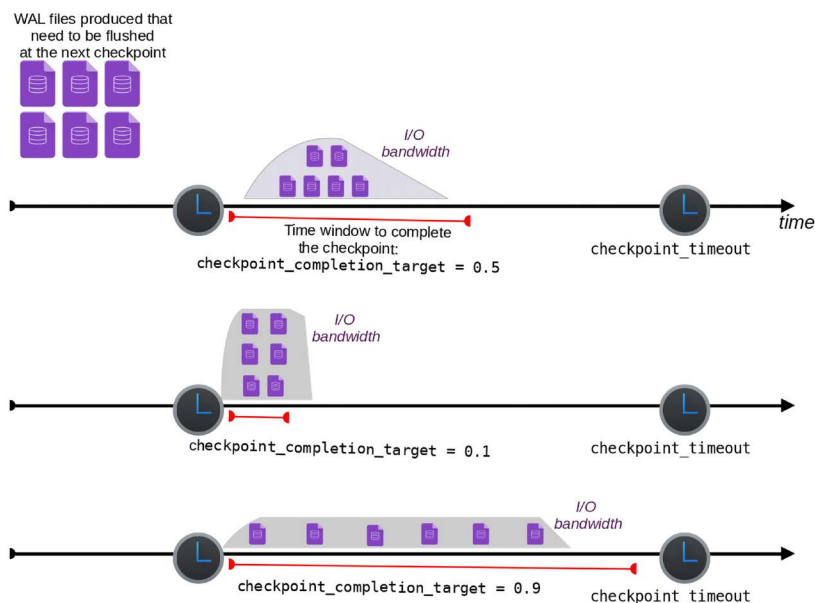


Figure 11.5: I/O bandwidth consumption

By default, `checkpoint_completion_target` is set to 0.9, meaning that the checkpoint will try to finish as slowly as possible, in order to avoid I/O spikes and resource consumption:

```
forumdb=> show checkpoint_completion_target;
checkpoint_completion_target
-----
0.9
(1 row)
```

Manually issuing a checkpoint

It is always possible for the cluster administrator to manually start a checkpoint process: the PostgreSQL statement `CHECKPOINT` starts all the activities that would normally be triggered at `checkpoint_timeout` or `max_wal_size`.

With the checkpoint being such an invasive operation, why should someone want to perform it manually? One reason could be to ensure that all the data on the disk has been synchronized, for example, before starting a streaming replication or a file-level backup.

In the following section, you will learn about the `VACUUM` process, the technique that allows PostgreSQL to reclaim unused space, removing no-longer-visible tuples.

VACUUM

In the previous sections, you learned how PostgreSQL exploits MVCC to store different versions of the same data (tuples) that different transactions can perceive, depending on their active snapshot. However, keeping different versions of the same tuples requires extra space with regard to the last active version, and this space could fill your storage sooner or later. To prevent that, and reclaim storage space, PostgreSQL provides an internal tool named `VACUUM`, the aim of which is to analyze stored tuple versions and remove the ones that are no longer perceivable.



Remember: a tuple is not perceivable (visible) when there are no more active transactions that can reference the version, which means having the tuple version within their snapshot. A not-perceivable tuple is often called a *dead tuple*, marking the fact that it is not required anymore in the database life cycle.

VACUUM can be an I/O-intensive operation, since it must reclaim and free disk space and, therefore, can be an invasive operation. For that reason, you are not supposed to run VACUUM by hand very frequently, and PostgreSQL also provides a background job, named autovacuum, which can run VACUUM for you depending on the current database activity.

The following subsections will show you both manual and automatic VACUUM.

Manual VACUUM

Manual VACUUM can be run against a single table, a subset of table columns, or a whole database, and the synopsis is as follows:

```
VACUUM [ FULL ] [ FREEZE ] [ VERBOSE ] [ ANALYZE ] [ table_and_columns [, ...] ]
```

There are three main versions of VACUUM that perform progressively more aggressive data refactoring:

- *Plain* VACUUM (the default) does a micro-space-reclaim, which means it throws away dead tuple versions but does not de-fragment the table, and therefore, the final effect is no space being reclaimed.
- VACUUM FULL performs a whole table rewrite, throwing away dead tuples and removing fragmentation, thus also aggressively reclaiming disk space.
- VACUUM FREEZE marks already consolidated tuples as frozen, preventing the xid wrap-around problem.

VACUUM cannot be executed within a transaction, nor a function or procedure. The extra options VERBOSE and ANALYZE provide a verbose output and perform a statistic update of the table contents (this is useful for performance gain) respectively.

In order to see the effects of VACUUM, let's build a simple example. First of all, ensure that autovacuum is set to off. If it's not, edit the \$PGDATA/postgresql.conf configuration file and set the parameter to off, and then restart the cluster. After that, inspect the size of the tags table:

```
forumdb=> SHOW autovacuum;
autovacuum
-----
off
(1 row)
```

```

forumdb=> SELECT relname, reltuples, relpages, pg_size_pretty( pg_
relation_size( 'tags' ) )
FROM pg_class WHERE relname = 'tags' AND relkind = 'r';
 relname | reltuples | relpages | pg_size_pretty
-----+-----+-----+-----
 tags   |         6 |         1 | 8192 bytes
(1 row)

```

As you can see, the table has only six tuples and occupies a single data page on disk, of 8 KB in size. Now, let's populate the table with about 1 million random tuples:

```

forumdb=> INSERT INTO tags( tag )
SELECT 'FAKE-TAG-#' || x
FROM generate_series( 1, 1000000 ) x;
INSERT 0 1000000

```

Since we have stopped autovacuum, PostgreSQL does not know the real size of the table, and therefore, we need to perform a manual ANALYZE to inform the cluster about the new data in the table:

```

forumdb=> ANALYZE tags;
ANALYZE
forumdb=> SELECT relname, reltuples, relpages, pg_size_pretty( pg_
relation_size( 'tags' ) )
FROM pg_class WHERE relname = 'tags' AND relkind = 'r';
 relname | reltuples | relpages | pg_size_pretty
-----+-----+-----+-----
 tags   | 1.00001e+06 |      6370 | 50 MB

```

It is now time to invalidate all the tuples we have inserted, for example, by overwriting them with an UPDATE (which, due to MVCC, will duplicate the tuples):

```

forumdb=> UPDATE tags SET tag = lower( tag ) WHERE tag LIKE 'FAKE%';
UPDATE 1000000

```

The table now still has around 1 million valid tuples, but the size has almost doubled because every tuple now exists in two versions, one of which is dead:

```

forumdb=> ANALYZE tags;
ANALYZE
forumdb=> SELECT relname, reltuples, relpages, pg_size_pretty( pg_
relation_size( 'tags' ) )

```

```
FROM pg_class WHERE relname = 'tags' AND relkind = 'r';
 relname | reltuples | relpages | pg_size_pretty
-----+-----+-----+-----
 tags    | 1.00001e+06 | 12739 | 100 MB
(1 row)
```

We have now built something that can be used as a test lab for VACUUM. If we execute plain VACUUM, every single data page will be freed of dead tuples but pages will not be reconstructed, so the number of data pages will remain the same, and the final table size on storage will be the same too:

```
forumdb=> VACUUM VERBOSE tags;
...
INFO: "tags": found 1000000 removable, 1000006 nonremovable row versions
in 12739 out of 12739 pages

VACUUM
forumdb=> ANALYZE tags;
ANALYZE

forumdb=> SELECT relname, reltuples, relpages, pg_size_pretty( pg_
relation_size( 'tags' ) )
FROM pg_class WHERE relname = 'tags' AND relkind = 'r';
 relname | reltuples | relpages | pg_size_pretty
-----+-----+-----+-----
 tags    | 1.00001e+06 | 12739 | 100 MB
(1 row)
```

VACUUM informs us that 1 million tuples can be safely removed, while 1 million (plus the original 6 tuples) cannot be removed because they represent the last active version. However, after this execution, the table size has not changed: all data pages have been de-fragmented internally, but no storage space is freed because the total number of pages did not change.

So, what is the aim of plain VACUUM? This kind of VACUUM provides new free space on every single page, so the table can essentially sustain 1 million new tuples without changing its own size. We can prove this by performing the same tuple invalidation we have already done:

```
forumdb=> UPDATE tags SET tag = upper( tag ) WHERE tag LIKE 'fake%';
UPDATE 1000000
forumdb=> ANALYZE tags;
```

```

ANALYZE
forumdb=> SELECT relname, reltuples, relpages, pg_size_pretty( pg_
relation_size( 'tags' ) )
FROM pg_class WHERE relname = 'tags' AND relkind = 'r';
  relname | reltuples | relpages | pg_size_pretty
-----+-----+-----+-----
  tags    | 1.00001e+06 |    12739 | 100 MB
(1 row)

```

As you can see, nothing has changed in the number of tuples, pages, and table size. Essentially, it went like this: we introduced 1 million new tuples in the beginning, then we updated all of them, making the 1 million become 2 million, then we used `VACUUM` on the table, lowering the number again to 1 million but leaving the free space already allocated so that the table was occupying space for 2 million but only half of that storage was full. After that, we created 1 million new tuple versions but the system did not need to allocate more space because there was enough free, even if scattered across the whole table.

On the other hand, `VACUUM FULL` not only frees the space within the table but also reclaims all such space, compacting the table to its minimum size. If we execute `VACUUM FULL` right now, at least 50 MB of data space will be reclaimed because 1 million tuples will be thrown away:

```

forumdb=> VACUUM FULL VERBOSE tags;
INFO:  vacuuming "public.tags"
INFO:  "tags": found 1000000 removable, 1000006 nonremovable row versions
in 12739 pages
DETAIL:  0 dead row versions cannot be removed yet.
CPU: user: 0.18 s, system: 0.61 s, elapsed: 1.03 s.
VACUUM
forumdb=> ANALYZE tags;
ANALYZE
forumdb=> SELECT relname, reltuples, relpages, pg_size_pretty( pg_
relation_size( 'tags' ) )
FROM pg_class WHERE relname = 'tags' AND relkind = 'r';
  relname | reltuples | relpages | pg_size_pretty
-----+-----+-----+-----
  tags    | 1.00001e+06 |     6370 | 50 MB
(1 row)

```